Adventist University of Central Africa (AUCA)

Master of Science in Big Data Analytics

Big Data Essentials: Course Project

8

Group



Catching Fraud in Real Time

An end-to-end Big Data pipeline that streams card transactions from HDFS through Kafka, scores them with a Spark MLlib model, stores every prediction in MySQL and shows them live on a Django dashboard.

Submitted by

Carlos HARERIMANA	101200
Kevin RUBERWA	101190
Ernest NDACYAYISENGA	101198
Felicien GAKUBA	101285
Florent Isheja MUNYAKAZI	101485

Lecturer

Dr. Kundan Kumar

Submitted

October 2026

Headline result

Random Forest, AUC 0.904, 82% of frauds caught

Stack: Hadoop HDFS 3.2.1, Apache Kafka (Confluent Platform 7.5.0), Apache Spark 3.5.3 with MLlib, MySQL 8.0, Django 4.2, all orchestrated with Docker.

Abstract

A stolen card is usually used within minutes, so a fraud check that runs overnight is already too late. This project builds the kind of system a card issuer needs instead: one that looks at every transaction the moment it happens, decides how suspicious it is, and puts the dangerous ones in front of an analyst while there is still time to act.

We generated a realistic dataset of **13.5 million card transactions (1.25 GB)** for customers in ten African and international cities, and stored it in the **Hadoop Distributed File System (HDFS)**. A **Kafka** producer reads the data directly from HDFS and replays it as a live stream. **Spark Structured Streaming** cleans each transaction and engineers five fraud signals, such as how far the card is from its owner's home and how much the purchase exceeds the customer's usual spending. A **Random Forest** model trained with **Spark MLlib** on historical data from HDFS then assigns every transaction a fraud probability and a risk level. Results are written to **MySQL** and displayed on a **Django** dashboard that refreshes every five seconds. The whole stack runs in **Docker** on a single 8 GB laptop.

On a held-out test set of 134,916 transactions the Random Forest reached an **AUC-ROC of 0.904** and caught **82% of frauds**, which is within sampling noise of the best score any model could achieve on this data. In a final live run, **10,000 transactions** were streamed and scored end to end: **69 of 95 frauds** were caught, and the share of transactions flagged for review (10.1%) matched the test set almost exactly. Along the way we found that our first dataset made good prediction mathematically impossible, measured why, and fixed it. That story, and what it teaches about data design, is part of our findings.

> **How to read this report**
>
> Chapters 1 to 6 follow the six items the project guidelines ask for, in the same order: Project Overview, System Architecture, Technologies Used, Version Information, Dependencies and Findings. Chapter 7 covers testing and deployment, and Chapter 8 is a step-by-step guide that lets anyone rebuild and run the system. Green boxes hold key findings, amber boxes hold lessons learned, and grey boxes explain technical ideas in plain words.

Contents

Abstract i

1 Project Overview 1

1.1 Why fraud detection has to be real time . . . 1

1.2 Project objectives . . . 1

1.3 The case study at a glance . . . 1

1.4 The workflow in seven steps . . . 2

1.5 Outcomes at a glance . . . 2

2 System Architecture 3

2.1 The big picture . . . 3

2.2 Following one transaction through the system . . . 3

2.3 Component by component . . . 4

2.3.1 Storage: HDFS . . . 4

2.3.2 Ingestion: the Kafka producer reads from HDFS . . . 4

2.3.3 Messaging: Kafka and ZooKeeper . . . 5

2.3.4 Processing: three Spark streaming stages . . . 5

2.3.5 Feature engineering . . . 5

2.3.6 Prediction: the MLlib pipeline . . . 6

2.3.7 Storage of results: MySQL . . . 6

2.3.8 Visualization: the Django dashboard . . . 7

2.4 Deployment on Docker . . . 7

3 Technologies Used 8

4 Version Information 10

5 Dependencies 11

5.1 JAR files . . . 11

5.2 Python packages . . . 12

5.3 Docker images and front-end assets . . . 12

6 Findings 13

6.1 What the data says about fraud . . . 13
6.2 Getting the data right: why we rebuilt the dataset . . . 14
6.3 Model comparison: Logistic Regression against Random Forest . . . 15
6.4 Where the models are right and wrong . . . 16
6.5 What drives the predictions . . . 16
6.6 Live results: does the model behave the same on the stream? . . . 17
6.7 The dashboard . . . 18
6.8 What it means for a bank . . . 19
6.9 Conclusions from the findings . . . 20

7 Testing and Deployment 21
7.1 How we tested . . . 21
7.2 Challenges and how we solved them . . . 21
7.3 Deployment . . . 22

8 Setup and Execution Guide 23
8.1 Requirements . . . 23
8.2 Project structure . . . 23
8.3 Step by step . . . 23
8.4 Starting over with a clean slate . . . 25
8.5 Troubleshooting . . . 26

9 Limitations and Future Work 27
9.1 Limitations . . . 27
9.2 Future work . . . 27

10 Conclusion 28

References 29

A Code Highlights 30
A.1 The shared feature recipe . . . 30
A.2 Scoring each micro-batch and writing it to MySQL . . . 30
A.3 The fraud formula of the dataset generator (v2) . . . 30

Chapter 1

Project Overview

1.1 Why fraud detection has to be real time

Card fraud is a race against the clock. Once criminals obtain card details, they tend to spend quickly, often with several purchases in a short burst, frequently far from where the real cardholder lives, and preferably on goods that are easy to resell, such as electronics and jewellery. A bank that reviews transactions in a nightly batch job discovers the damage the next morning. A bank that scores every transaction as it arrives can block the card, or call the customer, before the second or third purchase goes through.

That is exactly the situation the project guidelines [1] describe: *real-time data processing, predictive analytics and web-based visualization*, combined into one working system. We chose card fraud detection as our case study because it is a problem where the "real time" part is not decoration: it is the whole point. It also connects directly to the banking and credit risk experience within our group.

1.2 Project objectives

Table 1.1 maps each objective from the guidelines to what we built.

Table 1.1: Project objectives and how this project meets them

Objective (from the guidelines)	What we built	Where
Real-time data processing	A Kafka producer streams transactions read from HDFS; Spark Structured Streaming processes them in 5-second micro-batches.	Chapters 2, 6
Prediction modeling	Logistic Regression and Random Forest trained with Spark MLlib on historical data in HDFS; the better model scores the live stream.	Sections 2.3, 6.3
Database integration	Every scored transaction, with its fraud probability and risk level, is written to a MySQL table.	Section 2.3
Web dashboard visualization	A Django dashboard with live figures, a high-risk alert feed and charts, refreshing every 5 seconds.	Section 6.7

1.3 The case study at a glance

We simulate a card issuer with **50,000 customers** whose cards are used in ten cities: Kigali, Nairobi, Kampala, Lagos, Johannesburg, Cairo, Dubai, London, New York and Paris. Every cus-

tomer has a home city. Most purchases happen at home, but about 8% happen elsewhere, as they would for a traveller. Each transaction record has 14 readable columns (Table 1.2). We deliberately avoided public datasets whose features are anonymised (for example, columns named V1 to V28), because readable features let us engineer meaningful signals and explain the model's decisions to a non-technical reader.

Table 1.2: The 14 columns of each transaction

Column	Meaning
`transaction_id`, `customer_id`	Unique transaction number; which of the 50,000 customers paid
`timestamp`	Date and time of the purchase (a 90-day window starting 1 January 2026)
`amount`	Purchase amount in US dollars
`merchant_category`	One of 12 categories: grocery, electronics, travel, jewelry, hotel, . . .
`device_type`, `card_present`	Mobile app, web browser, POS terminal or ATM; whether the physical card was used
`customer_age`, `customer_tenure_days`	Age of the customer; how long they have been a customer
`avg_txn_amount_last_30d`	The customer's average purchase over the last 30 days
`transactions_last_1h`	How many purchases the card made in the previous hour
`distance_from_home_km`	Distance between the purchase location and the customer's home city
`txn_city`	City where the purchase happened
`is_fraud`	The true label: 1 if the transaction is fraudulent, 0 otherwise

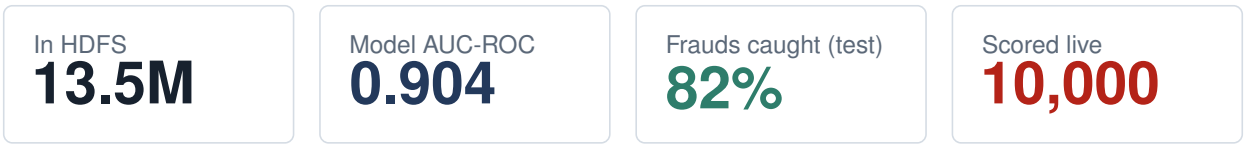
1.4 The workflow in seven steps

The guidelines define seven steps, and our system follows them one to one (Figure 1.1).



Figure 1.1: The seven project steps from the guidelines, as implemented

1.5 Outcomes at a glance



In short: the complete pipeline works from end to end, the model performs at the ceiling of what the data allows, and its behaviour on the live stream matches what it showed during testing. Chapter 6 explains each of these results in detail.

Chapter 2

System Architecture

2.1 The big picture

Figure 2.1 shows the complete system. It has two paths that share the same storage. The **training path** (bottom) runs once, in batch: Spark reads the historical transactions from HDFS, trains the model and saves it back into HDFS. The **streaming path** (top) runs continuously: transactions flow from HDFS through Kafka into Spark, are scored by the saved model, land in MySQL and appear on the dashboard.

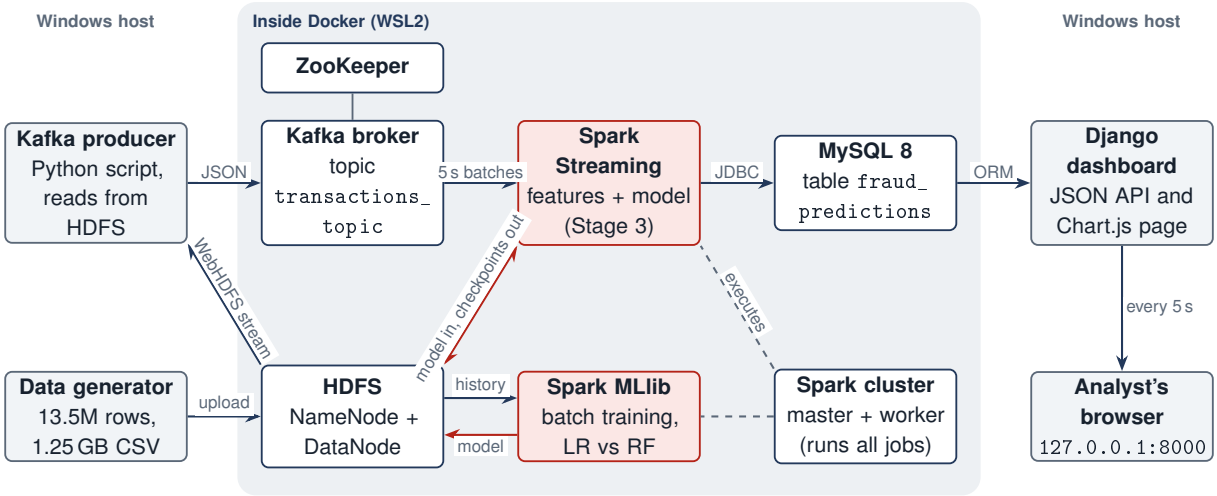


Figure 2.1: System architecture. White components run inside Docker, grey ones on the Windows host, and red ones are where machine learning happens.

> **In plain words**
>
> Think of HDFS as the bank's archive, Kafka as a conveyor belt that brings transactions in one by one, Spark as the inspector standing at the belt with a trained eye (the model), MySQL as the logbook where every decision is written down, and Django as the screen on the fraud analyst's desk.

2.2 Following one transaction through the system

The easiest way to understand the architecture is to follow a single transaction. We picked a real one from our live run: **transaction 2933**, a $468.79 electronics purchase in Nairobi, made 11,842 km from the cardholder's home and 3.1 times larger than the customer's usual spending. It is a genuine fraud in the dataset.

Stored
The transaction is one line in `transactions.csv`, kept as blocks inside HDFS at `/fraud_project/data/`.

Read and published
The producer streams the file from HDFS over WebHDFS, converts the row to JSON and publishes it to the Kafka topic `transactions_topic`.

Picked up
Spark Structured Streaming polls Kafka every 5 seconds and receives the message inside a micro-batch of up to 2,000 transactions.

Cleaned and enriched
Spark parses the JSON, checks for missing or invalid values, and adds five engineered features. Here: spend ratio 3.1, far from home = 1, high-risk merchant = 1.

Scored
The Random Forest pipeline (loaded from HDFS) returns a fraud probability of 0.98. Because it is above 0.80, the risk level is HIGH.

Logged
The row, its features, probability, prediction and risk level are appended to the MySQL table `fraud_predictions`.

Shown
Within the next dashboard refresh, the alert appears at the top of the high-risk feed with the reasons "11,842 km from home", "3.1× usual spend" and "electronics purchase".

Figure 2.2: The journey of transaction 2933, from storage to the analyst's screen

2.3 Component by component

2.3.1 Storage: HDFS

The dataset lives in HDFS, served by two containers: a **NameNode**, which keeps the file-system map, and a **DataNode**, which stores the actual 128 MB blocks. Because this is a single-machine cluster, the replication factor is set to 1. HDFS holds three things for the project:

```
/fraud_project/
  data/transactions.csv          <- the 1.25 GB dataset (13.5 million rows)
  models/fraud_model/            <- the trained Random Forest pipeline (metadata + stages)
  checkpoints/stage3/            <- streaming progress, so the job can resume after a restart
```

2.3.2 Ingestion: the Kafka producer reads from HDFS

The guidelines require that the producer reads its data from HDFS, not from a local copy. Our producer (`producer/kafka_producer.py`) runs on Windows and uses **WebHDFS**, the REST interface built into HDFS, so no Hadoop client needs to be installed. Reading a file takes two hops: the NameNode answers with a redirect to the DataNode that holds the blocks, and the producer then streams the file from the DataNode line by line. The 1.25 GB file is therefore never loaded into memory.

Each row becomes a JSON message. A small pause between messages (10 ms by default) turns the static file into a realistic stream of roughly 85 transactions per second on our laptop. Two

options make demonstrations repeatable: `--max-rows` limits how many transactions are sent, and `--start-row` continues from a given position so that each run streams fresh transactions.

2.3.3 Messaging: Kafka and ZooKeeper

Kafka receives the messages on the topic `transactions_topic`, and ZooKeeper coordinates the broker. The broker advertises two addresses: `localhost:29092` for programs on Windows (the producer) and `kafka:9092` for programs inside the Docker network (Spark). This dual-listener setup is a common source of confusion, so Table 2.1 spells it out.

Table 2.1: Who connects to Kafka, and how

Client	Address	Why
Kafka producer	`localhost:29092`	Runs on Windows, outside Docker, so it uses the port published to the host.
Spark consumer	`kafka:9092`	Runs inside Docker, where containers find each other by name.

2.3.4 Processing: three Spark streaming stages

We built the Spark consumer in three stages, each tested before the next was added (Table 2.2). Only Stage 3 is needed in production, but keeping the earlier stages made debugging much easier and documents how the system was built.

Table 2.2: The three Spark Structured Streaming consumers

Stage	What it does	Output
Stage 1	Reads the Kafka topic and parses each JSON message into typed columns.	Console: raw transactions, to verify the stream works.
Stage 2	Adds cleaning (drops malformed rows and non-positive amounts, parses timestamps) and the engineered features.	Console: transactions with features, verified by hand.
Stage 3	Adds the trained model, fraud probability, risk level, and writes each micro-batch to MySQL with checkpointing.	MySQL table `fraud_predictions`.

2.3.5 Feature engineering

Raw columns do not describe fraud directly. The engineered features in Table 2.3 translate fraud-analyst intuition into numbers a model can use.

Table 2.3: Engineered features and the fraud behaviour each one captures

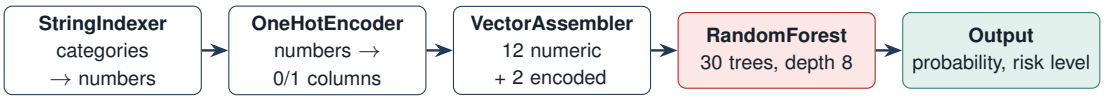
Feature	How it is computed	Fraud behaviour it captures
`amount_to_avg_ratio`	amount ÷ 30-day average spend	A thief spends far more than the real owner normally does.
`is_far_from_home`	1 if distance $>$ 500 km	Stolen card details are often used in another city or country.
`is_high_velocity`	1 if 3 or more purchases in the last hour	Fraudsters make several quick purchases before the card is blocked.
`is_high_risk_merchant`	1 for jewelry or electronics	Goods that are easy to resell for cash.
`hour_of_day`	hour of the timestamp	Captures time-of-day patterns in spending.
`card_present_int`	card_present as 0 or 1	Card-not-present purchases (online, in-app) are easier to commit with stolen details.

A design choice that matters: one feature recipe

The exact same feature code is used in the training script and in the streaming consumer. If training and live scoring computed features even slightly differently, the model would see different inputs in production than it learned from, a problem known as *training/serving skew*. Sharing one recipe rules it out.

2.3.6 Prediction: the MLlib pipeline

The model is saved as a complete MLlib `PipelineModel`, not just a classifier. The pipeline contains every preparation step, so the streaming job simply loads it and calls `transform`:



The fraud probability is then mapped to three risk levels that work as a triage queue for analysts: **HIGH** ($\geq$ 0.80), **MEDIUM** (0.50 to 0.80, the model's default decision threshold) and **LOW** ($<$ 0.50).

2.3.7 Storage of results: MySQL

Spark writes each micro-batch to MySQL through JDBC using `foreachBatch`. Every row keeps the original transaction data, the engineered features, the fraud probability, the prediction, the risk level, the true label (so live accuracy can be measured), the Spark batch number and a processing timestamp. Indexes on time, risk level and prediction keep the dashboard queries fast. Streaming progress is checkpointed in HDFS, so if the job is stopped and restarted it continues from the last processed Kafka offset instead of starting over.

2.3.8 Visualization: the Django dashboard

The dashboard is a small Django project with one app, `monitor`. Its data model maps the MySQL table with `managed = False`, which tells Django to read the table but never to create or change it: Spark owns the table, Django only reads it. One view renders the page, and a second view, `/api/summary/`, returns all figures as JSON. The page calls this endpoint every five seconds and redraws its numbers, alert feed and Chart.js charts, which is what makes the dashboard update live while transactions stream in.

2.4 Deployment on Docker

All Big Data services run as Docker containers defined in a single `docker-compose.yml` (Table 2.4). The producer and the Django dashboard run directly on Windows.

Table 2.4: Containers, images and ports

Container	Image	Role	Ports
`namenode`	`bde2020/hadoop-namenode: 2.0.0-hadoop3.2.1-java8`	HDFS metadata, WebHDFS	`9870`, `9000`
`datanode`	`bde2020/hadoop-datanode: 2.0.0-hadoop3.2.1-java8`	HDFS data blocks	`9864`
`zookeeper`	`confluentinc/cp-zookeeper:7.5.0`	Kafka coordination	`2181`
`kafka`	`confluentinc/cp-kafka:7.5.0`	Message broker	`9092`, `29092`
`spark-master`	`bde-spark:3.5.3 (custom)`	Spark master, runs drivers	`7077`, `8080`
`spark-worker`	`bde-spark:3.5.3 (custom)`	Spark executor (1 GB, 2 cores)	`8081`
`mysql`	`mysql:8.0`	Prediction storage	`3307`

Three design decisions make the deployment robust:

- **A custom Spark image.** `bde-spark:3.5.3` extends the official Apache Spark image with numpy and pandas, which MLlib's Python side needs. They are part of the image, so they survive any container reset.
- **Named volumes for all state.** HDFS, MySQL, Kafka and ZooKeeper each store their data in a Docker volume, so restarting or rebuilding containers never loses the dataset, the model or the predictions.
- **Shared project folders.** The `consumer`, `models` and `jars` folders on Windows are mounted into the Spark containers. Code edited on Windows is immediately visible to Spark, and training results appear directly in the Windows project folder.

The laptop has 8 GB of RAM in total. Docker runs in WSL2 with 3 GB of memory and 2 processors, and the Spark worker is limited to 1 GB and 2 cores. Section 7.2 explains how we arrived at these numbers.

Chapter 3

Technologies Used

The guidelines name six technologies. We used all six, plus Docker to run them together on one machine. For each one, this chapter explains what it is, what it does in our project and why it suits the job.

Hadoop Distributed File System (HDFS) — Storage

What it is. A file system that splits large files into blocks and spreads them across many machines, so storage grows by adding machines instead of buying bigger ones [2].
In our project. Holds the 1.25 GB dataset, the trained model and the streaming checkpoints. The Kafka producer reads from it over WebHDFS, and Spark reads from it for training and writes the model back to it.
Why it fits. It is the standard storage layer of the Hadoop ecosystem, and Spark reads it natively (`hdfs://namenode:9000/...`).

Apache Kafka — Messaging and streaming

What it is. A distributed log that receives streams of messages from producers and keeps them, in order, for consumers to read at their own pace [3].
In our project. The topic `transactions_topic` carries every transaction from the producer to Spark as a JSON message. ZooKeeper coordinates the broker.
Why it fits. Kafka decouples the two sides: the producer can send faster than Spark processes, or Spark can be restarted, and no transaction is lost because Kafka keeps them and Spark remembers its position.

Apache Spark with PySpark — Processing

What it is. A distributed processing engine. PySpark is its Python interface, and Structured Streaming is its API for processing data streams as a continuously growing table [4, 5].
In our project. A standalone cluster (one master, one worker) runs two kinds of job: the batch training job and the streaming consumer that cleans, enriches and scores transactions in 5-second micro-batches.
Why it fits. The same engine and the same DataFrame code serve both batch and streaming, which is how we share one feature recipe between training and live scoring.

Spark MLlib — Machine learning

What it is. Spark's built-in machine learning library, with algorithms that train on distributed data and pipelines that bundle data preparation with the model [6].
In our project. Trains Logistic Regression and Random Forest classifiers on historical data from HDFS, compares them, and saves the winning pipeline to HDFS for the streaming job to load.

Why it fits. Training data never has to leave the cluster, and a saved `PipelineModel` can be applied to a streaming DataFrame directly.

MySQL — Results database

What it is. A widely used open-source relational database.
In our project. Stores every scored transaction in the table `fraud_predictions`: the original data, the engineered features, fraud probability, prediction, risk level and the true label.
Why it fits. The dashboard needs fast, flexible queries (counts, groupings, latest alerts), which is exactly what a relational database with indexes provides. The guidelines allow MySQL or AWS RDS; we ran MySQL locally (Chapter 9 discusses moving to AWS RDS).

Django — Web dashboard

What it is. A Python web framework with a built-in database layer (the ORM), URL routing and templates [11].
In our project. Serves the fraud monitor page and a JSON endpoint that aggregates the MySQL data. The page uses Chart.js to draw its charts and refreshes itself every five seconds.
Why it fits. Its ORM turns the dashboard's questions ("how many HIGH alerts per city?") into short, readable Python instead of hand-written SQL.

Docker and Docker Compose — Supporting tool

What it is. Containers package a program with everything it needs; Compose starts a group of containers from one configuration file.
In our project. One `docker-compose.yml` starts all seven Big Data services with their networking, storage volumes and resource limits.
Why it fits. Installing Hadoop, Kafka and Spark by hand on Windows is slow and fragile. With Compose, the whole cluster starts with one command and behaves the same on any machine.

Chapter 4

Version Information

Table 4.1 lists the exact version of every tool used. Versions matter in the Big Data world because components must match: for example, the Kafka connector for Spark must be built for the same Spark and Scala versions as the cluster (Chapter 5).

Table 4.1: Version of every tool and platform used in the project

Tool	Version	Notes
Big Data stack		
Hadoop (HDFS)	**3.2.1**	Images `bde2020/hadoop-namenode` and `-datanode`, tag `2.0.0-hadoop3.2.1-java8`
Apache Kafka	**Confluent Platform 7.5.0**	Image `confluentinc/cp-kafka:7.5.0`, based on Apache Kafka 3.5
Apache ZooKeeper	**Confluent Platform 7.5.0**	Image `confluentinc/cp-zookeeper:7.5.0`
Apache Spark	**3.5.3**	Standalone cluster, official `apache/spark:3.5.3` base image
PySpark	**3.5.3**	Bundled with Spark
Spark MLlib	**3.5.3**	Bundled with Spark (`pyspark.ml`)
Scala	**2.12**	The Scala version Spark 3.5.3 is built with
Java (Spark containers)	**11.0.24**	OpenJDK, from the Spark image
MySQL	**8.0**	Official image `mysql:8.0`
Application		
Python (Spark containers)	**3.8**	Runs the PySpark driver and executors
Python (Windows host)	**3.12**	Runs the data generator, Kafka producer and Django
Django	**4.2 (LTS)**	Web dashboard
Chart.js	**4.4.1**	Dashboard charts, loaded from the jsDelivr CDN
Platform		
Docker Desktop	**4.92.0**	With Docker Compose
WSL2 kernel	**6.18.33.2**	Linux virtual machine behind Docker Desktop
Operating system	**Windows 11 (build 22621)**	Laptop with 8 GB RAM

How to check a version yourself

Most versions can be read directly from the running system, for example `docker exec spark-master /opt/spark/bin/spark-submit --version` for Spark and Scala, `docker exec mysql mysql --version` for MySQL, and `python -m django --version` for Django. The Spark, Java and WSL2 versions above were read from Spark's own start-up log.

Chapter 5

Dependencies

5.1 JAR files

Spark does not include a Kafka connector or a MySQL driver out of the box, so five JAR files (Java libraries) are supplied when a job is submitted (Table 5.1).

Table 5.1: JAR files added to Spark jobs

JAR file	Purpose	Used by
`spark-sql-kafka-0-10_2.12-3.5.3.jar`	Kafka source for Spark Structured Streaming	Stages 1–3
`spark-token-provider-kafka-0-10_2.12-3.5.3.jar`	Kafka authentication support, required by the connector above	Stages 1–3
`kafka-clients-3.4.1.jar`	The Kafka Java client the connector is built on	Stages 1–3
`commons-pool2-2.11.1.jar`	Object pooling, used to reuse Kafka consumers	Stages 1–3
`mysql-connector-j-8.0.33.jar`	JDBC driver for writing predictions to MySQL	Stage 3

> **What the "_2.12-3.5.3" in the file names means**
>
> Spark is written in Scala, and a library compiled for one Scala version does not work with another. The suffix `_2.12` says the connector was built for Scala 2.12, and `3.5.3` says it was built for Spark 3.5.3. Both match our cluster exactly. Using a connector built for another Spark or Scala version is one of the most common causes of cryptic errors in Spark projects.

The JARs are stored in the project's `jars` folder, which is mounted into the Spark containers at `/tmp/jars`, and passed to `spark-submit` with the `--jars` option [10]. Spark can also download connectors automatically with `--packages`, and that is what we used at first. We switched after the Spark container temporarily lost access to the Maven repository and a job failed before it even started; with local JARs, jobs no longer depend on an internet connection (Section 7.2).

Several further libraries the connector needs are already part of the Spark distribution, so they did not have to be added: `hadoop-client-api` and `hadoop-client-runtime` 3.3.4, `lz4-java` 1.8.0, `snappy-java` 1.1.10.5, `slf4j-api` 2.0.7, `jsr305` 3.0.0 and `commons-logging` 1.1.3.

5.2 Python packages

Table 5.2: Python packages and where they run

Package	Version	Runs in	Why it is needed
`numpy`	1.24.4	Spark image	Required by `pyspark.ml`. Kept below 2.0 for compatibility with Spark 3.5.
`pandas`	2.0.3	Spark image	Imported by `pyspark.ml.functions`, which provides `vector_to_array` to read the fraud probability.
`kafka-python`	3.0.10	Windows host	Kafka producer client.
`requests`	2.34.2	Windows host	Streams the dataset from HDFS over WebHDFS.
`numpy, pandas`	recent	Windows host	Used by the dataset generator.
`Django`	4.2	Windows host	Web dashboard framework.
`PyMySQL`	≥ 1.1	Windows host	Pure-Python MySQL driver for Django (no C compiler needed on Windows).
`cryptography`	≥ 41	Windows host	Needed by PyMySQL for MySQL 8's default secure login method.

The Spark image is built from a two-line Dockerfile, so the packages are installed once, at build time, and are present every time the containers start:

```
FROM apache/spark:3.5.3
USER root
RUN pip3 install --no-cache-dir "numpy<2" "pandas<2.1"
USER spark
```

5.3 Docker images and front-end assets

The seven containers use the images listed in Table 2.4. The dashboard page also loads Chart.js 4.4.1 from the jsDelivr CDN and the Public Sans typeface from Google Fonts, so the browser showing the dashboard needs internet access; everything else runs offline.

Chapter 6

Findings

This chapter presents what we learned, in the order a reader would ask: what does the data say about fraud (6.1), was the data good enough to learn from (6.2), how well do the models work (6.3 to 6.5), does the model behave the same way on the live stream (6.6), what does the analyst see (6.7), and what does it all mean for a bank (6.8).

6.1 What the data says about fraud

Before any model, we measured how often fraud occurs when each warning signal is present, across all 13.5 million transactions. Overall, **122,721 transactions are fraudulent, a rate of 0.91%**, close to the levels card issuers see in practice. Figure 6.1 shows how strongly each signal raises that rate.

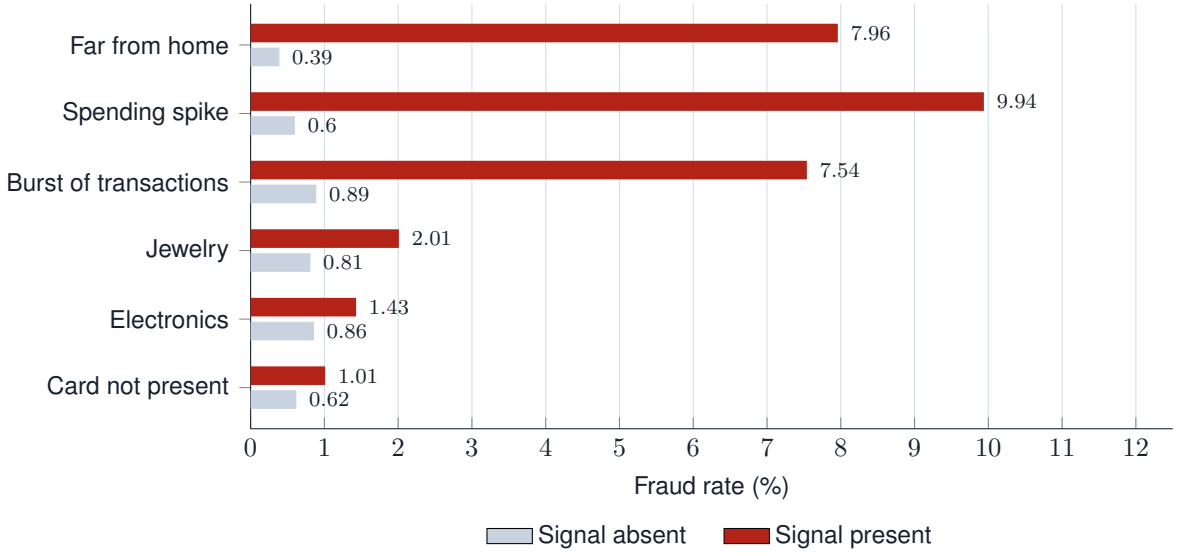


Figure 6.1: Fraud rate with and without each warning signal, over all 13.5 million transactions. A purchase more than 500 km from home is 20 times more likely to be fraud; a purchase above three times the customer's usual spending is 17 times more likely. (For "card not present", the absent bar is card-present purchases.)

The strongest finding, however, is what happens when signals appear *together* (Figure 6.2).

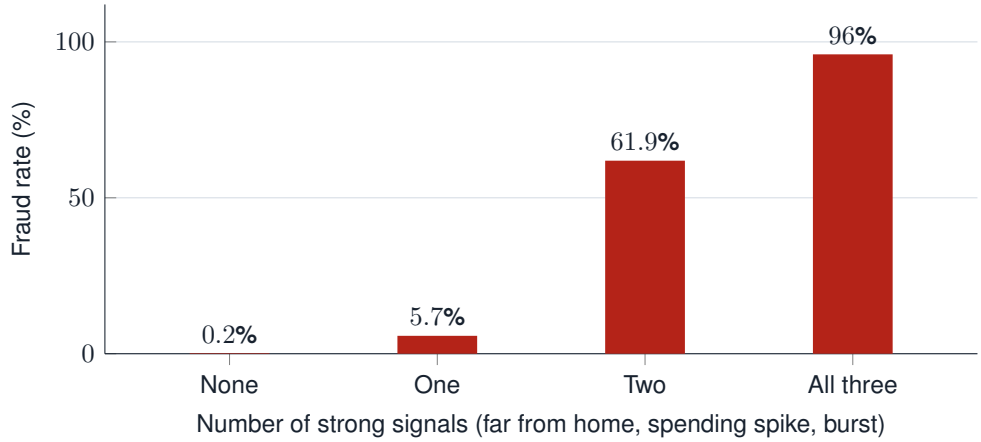


Figure 6.2: Fraud rate by number of strong signals. Transactions: 12.1 million with none, 1.35 million with one, 35,684 with two and 99 with all three.

Key finding 1: one red flag is a hint, two are a verdict

A single strong signal raises the fraud rate from 0.19% to 5.7%, which still means 94 out of 100 such transactions are innocent: travellers and big one-off purchases look like this too. Two signals together raise it to 62%, and all three to 96%. A fraud system that only uses fixed rules ("block anything far from home") would drown in false alarms; what matters is the *combination* of signals, which is exactly what a machine learning model learns.

Fraudulent purchases are also larger on average ($245 against $156 for legitimate ones), and card-not-present purchases (online and in-app) are 1.6 times riskier than card-present ones.

6.2 Getting the data right: why we rebuilt the dataset

Our first dataset (v1) produced a disappointing model: an AUC-ROC of only 0.63, not far above the 0.5 of random guessing. Instead of tuning the model, we asked a different question: *what is the best score any model could possibly achieve on this data?* Because we generated the data ourselves, we know the exact probability of fraud for every transaction, so we could compute the score of a "perfect" model that knows the true formula.

Table 6.1: The best score any model could reach, measured on 1 million transactions from each dataset version

Dataset	Fraud rate	Best possible AUC-ROC	Best possible AUC-PR	Our best model
v1 (first)	5.6%	0.611	0.095	0.63 (at the ceiling)
v2 (final)	0.91%	0.898	0.232	**0.904 and 0.223**

The answer was clear: in v1, every transaction had a base fraud chance of about 5% no matter what it looked like, so most frauds were pure chance and no model could predict them. In v2 we kept the same columns and signals but made normal transactions genuinely low risk and let each warning signal raise the risk sharply, which is how real fraud behaves. The same pipeline then reached an AUC-ROC of 0.904.

Lesson learned: measure the ceiling before tuning the model

When a model performs poorly, the problem is often the data, not the algorithm. Estimating the best achievable score first saved us from hours of pointless model tuning, and it gives an honest yardstick: on v2, our Random Forest reaches 96% of the best possible AUC-PR (0.223 of 0.232). On real data the true formula is unknown, but the same principle applies: check whether the features contain the signal before blaming the model.

6.3 Model comparison: Logistic Regression against Random Forest

Both models were trained on a random 5% sample of the HDFS dataset (674,809 transactions, 6,164 of them fraud), split 80/20 into 539,893 training and 134,916 test transactions. Because fraud is rare, each fraudulent transaction was given a higher weight during training [9]; otherwise a model could reach 99% accuracy by calling everything legitimate and catching nothing.

Table 6.2: Test-set results (134,916 transactions, 1,289 of them fraud). Best value in each row in bold.

Metric	Logistic Regression	Random Forest
AUC-ROC	0.895	**0.904**
AUC-PR (area under the precision-recall curve)	0.155	**0.223**
Recall (share of frauds caught)	77.3%	**82.4%**
Precision (share of flags that are real fraud)	6.8%	**7.8%**
F1 score	0.124	**0.142**
Accuracy (misleading on rare events, shown for completeness)	89.6%	**90.5%**
Training time on our laptop	66.0 s	**54.7 s**

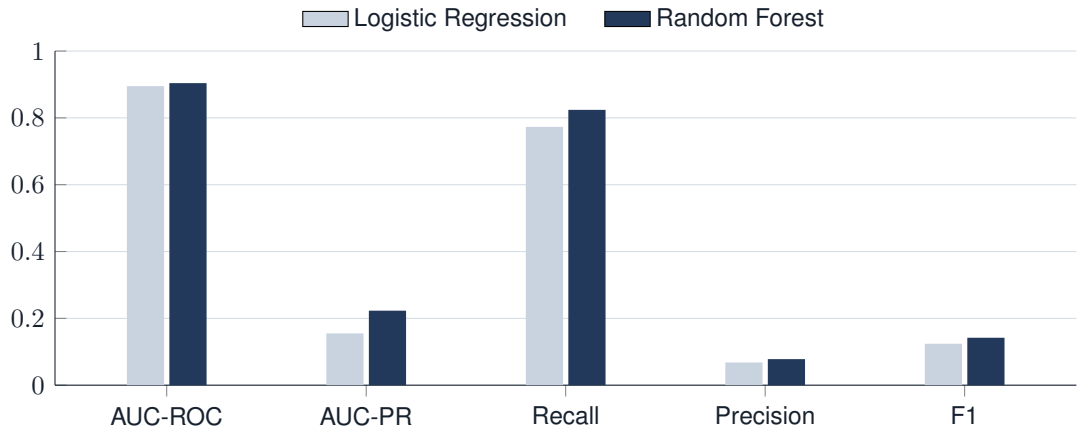


Figure 6.3: Random Forest wins on every metric (exact values in Table 6.2)

Key finding 2: the Random Forest wins, most clearly on AUC-PR

The two models are close on AUC-ROC, but the Random Forest is 44% better on AUC-PR (0.223 against 0.155), the metric that matters most when the positive class is rare [8]. For

context, a model that guesses randomly would score an AUC-PR equal to the fraud rate, about 0.01; the Random Forest is 23 times better than that. The Random Forest [7] also captures signal *combinations* naturally, because each tree splits on one signal and then another, which matches Key finding 1. It was therefore saved as the production model.

Why not simply use accuracy?

Only about 1 transaction in 100 is fraud. A "model" that approves everything is 99% accurate and completely useless. That is why we judge models by recall (how many frauds are caught), precision (how many alerts are real), and the AUC measures, which summarise the trade-off between the two across all possible thresholds.

6.4 Where the models are right and wrong



Figure 6.4: Confusion matrices on the test set. Compared with Logistic Regression, the Random Forest catches 66 more frauds and raises 1,099 fewer false alarms.

6.5 What drives the predictions

Figure 6.5 shows which inputs the Random Forest relies on most. Distance from home and the spending ratio together account for 85% of the model's decisions, matching the signal analysis in Figure 6.1.

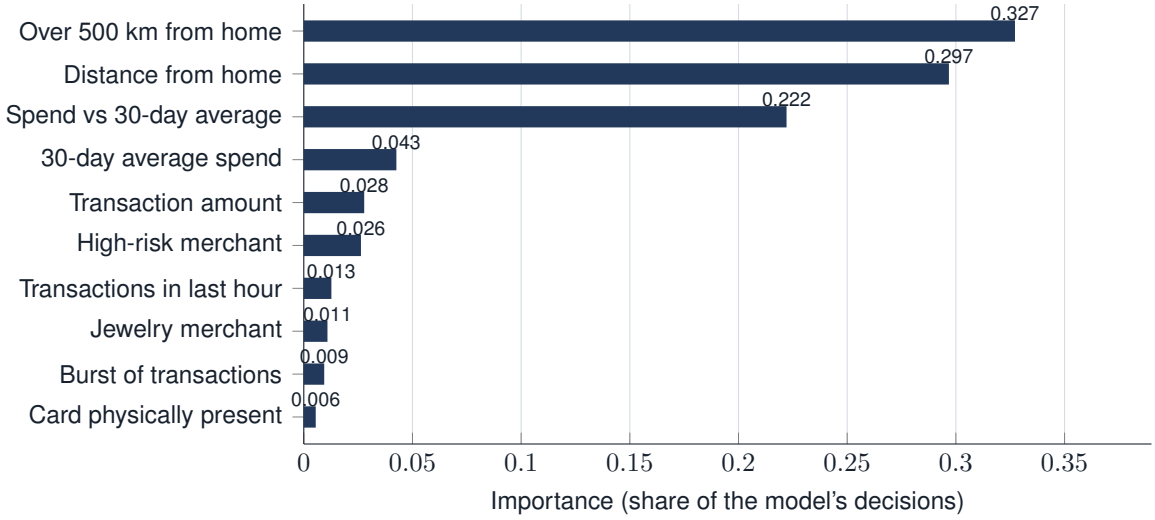


Figure 6.5: Top 10 features by Random Forest importance

Two observations stand out. First, the **engineered features lead the ranking**: the 500 km flag and the spend-versus-average ratio, both created in the feature engineering step, are two of the top three. The raw transaction amount on its own matters much less than the amount *relative to the customer's habits*. Second, the burst-of-transactions signal ranks low even though it is a strong fraud indicator (Figure 6.1). The reason is rarity: only 0.36% of transactions have a burst, so it decides few cases overall, but when it appears it matters a great deal.

Key finding 3: the model catches almost every fraud that can be caught

81.4% of all frauds in the dataset carry at least one strong warning signal; the remaining 18.6% look exactly like normal purchases. The Random Forest's recall is 82.4%. In other words, the model catches almost precisely the frauds that leave a trace, and misses mainly the ones that leave none. To catch more, a bank would need new kinds of information (device fingerprints, merchant history, customer contact), not a cleverer algorithm.

6.6 Live results: does the model behave the same on the stream?

A model can look good in testing and behave differently once deployed. For the final demonstration we cleared all previous results and streamed **10,000 transactions** from HDFS through Kafka and Spark into MySQL, then compared the live figures with the test set (Table 6.3).

Table 6.3: Test set compared with the live stream

Measure	Test set	Live stream
Transactions scored	134,916	10,000
Real frauds among them	1,289	95
Share of transactions flagged for review	10.2%	10.1%
Frauds caught (recall)	82.4%	72.6% (69 of 95)
Precision of flags	7.8%	6.8%

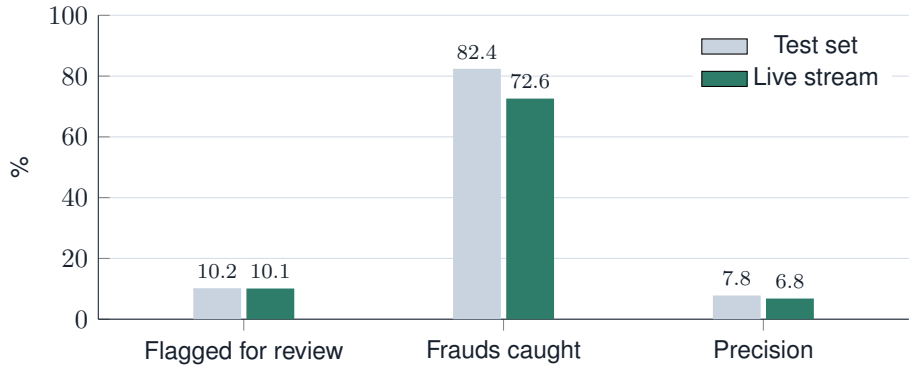


Figure 6.6: The model flags the same share of transactions live as in testing

The share of transactions flagged is almost identical (10.2% against 10.1%), which shows that the streaming pipeline computes features exactly as the training pipeline did: the shared feature recipe works. Live recall is lower (72.6% against 82.4%), but it rests on only 95 frauds, so every single fraud moves it by about one percentage point and its statistical margin of error is roughly

±9 points. The live result is consistent with the test result, and a longer stream would narrow the gap.

Key finding 4: training and live scoring agree

The pipeline behaves in production as it did in testing: same flag rate, similar precision, and recall within the expected range for a small sample. This is the best evidence that the streaming path (Kafka, Spark features, saved model, MySQL) is implemented correctly end to end.

6.7 The dashboard

Figures 6.7 to 6.9 show the dashboard at the end of the live run. The top of the page (Figure 6.7) summarises the stream: 10,000 transactions scored, 1,008 flagged, 552 high-risk alerts, 69 of 95 frauds caught and $183.5K worth of transactions under review.

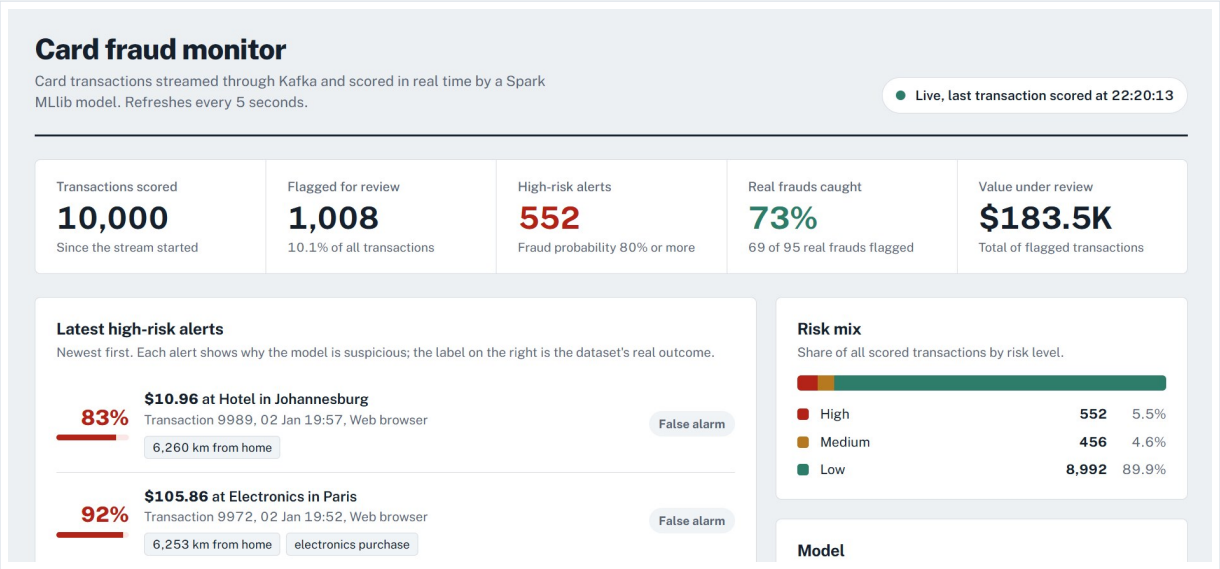


Figure 6.7: Dashboard: live summary figures, the high-risk alert feed and the risk mix

Each alert explains itself (Figure 6.8): the fraud probability, the reasons it was flagged and, since our data is labelled, whether it really was fraud. The Model panel shows the Random Forest's test results and what it relies on most, in plain language.

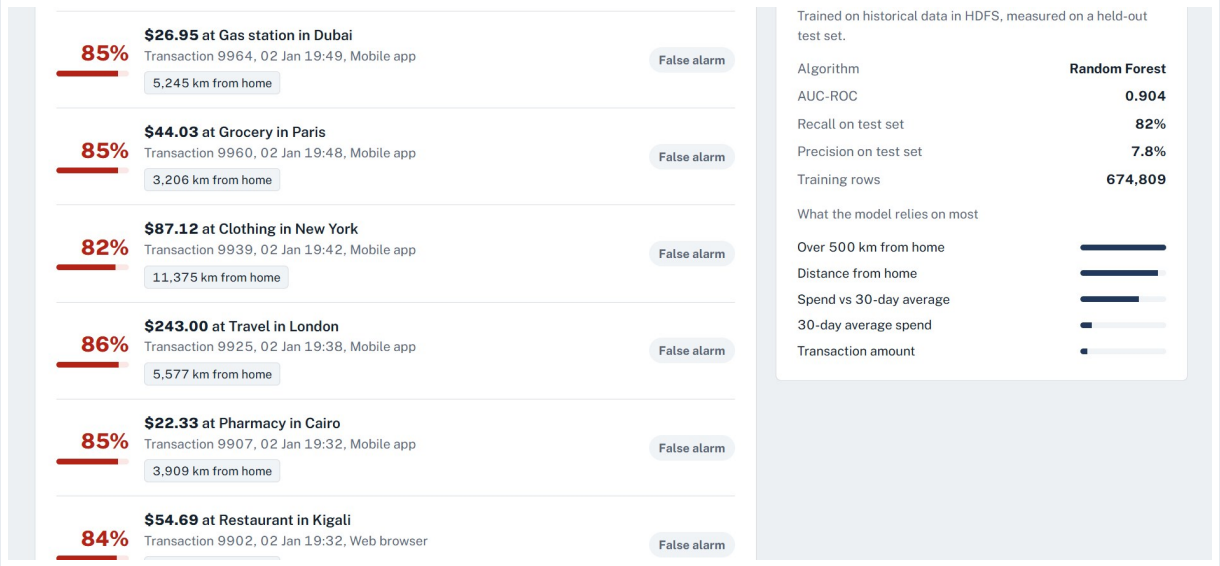


Figure 6.8: Dashboard: alert details with their reasons, and the Model panel

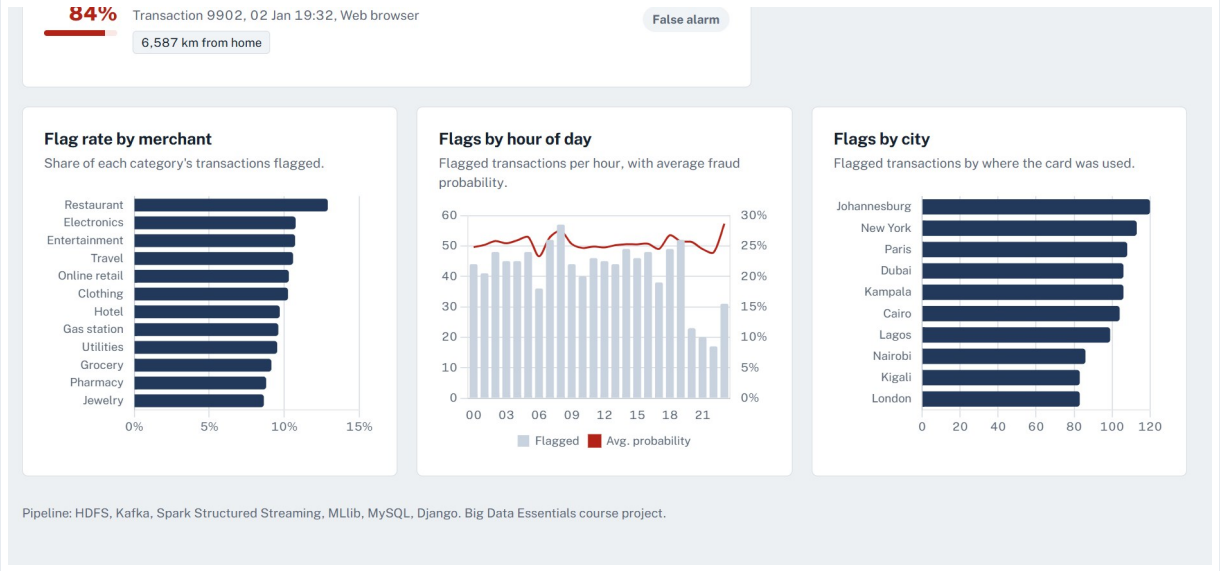


Figure 6.9: Dashboard: flag rate by merchant category, flags by hour of day and flags by city

6.8 What it means for a bank

Seen through a credit risk lens, the numbers tell a practical story. Out of every 10,000 transactions, the model sends about 1,000 for review and catches roughly three quarters to four fifths of the fraud among them. Precision of 7 to 8% means an analyst reviews about 12 alerts for each real fraud. That sounds high, but the costs are very unequal: a review takes minutes and a phone call, while a missed fraud costs the full amount and damages customer trust. Most card issuers accept a similar trade-off.

The risk levels make the workload manageable. The 552 HIGH alerts (probability of 80% or more) form the urgent queue, where a card could be blocked immediately; the 456 MEDIUM alerts can be handled with a lighter check, such as a confirmation text message to the customer. The thresholds are business decisions, not technical ones: a bank that wants fewer false alarms can raise them

and accept catching fewer frauds, and the dashboard would show the effect immediately.

6.9 Conclusions from the findings

1. **Combinations of signals, not single signals, identify fraud.** Two strong signals together mean a 62% fraud rate; one alone means 5.7%.
2. **Engineered features are the most valuable inputs.** The distance flag and the spend ratio, both created in our feature engineering step, carry most of the model's predictive power.
3. **Random Forest is the better model** for this problem, especially on AUC-PR, and it reaches the theoretical ceiling of the data.
4. **Data quality sets the limit.** Our first dataset capped any model at an AUC of 0.61; measuring that ceiling was more useful than any amount of model tuning.
5. **The pipeline is consistent end to end.** The live stream reproduces the test set's behaviour, which confirms that the real-time path is correctly implemented.

Chapter 7

Testing and Deployment

7.1 How we tested

We built the system one component at a time and did not move on until the current one was proven to work. Every new script was also run first on a small local sample of the data before it touched the full cluster, which caught several errors cheaply. Table 7.1 lists the main tests and their outcomes.

Table 7.1: End-to-end tests

Component	Test	Evidence	Result
HDFS	Upload the dataset and list it	`hdfs dfs -ls` shows `transactions.csv`, 1.2 G	**Pass**
Producer	Stream 5,000 rows read from HDFS	"Reading from HDFS", 5,000 messages sent in 59 s	**Pass**
Stage 1	Parse Kafka messages	Console batches show all 14 columns with correct types	**Pass**
Stage 2	Verify features by hand	e.g. 45.21 ÷ 40.57 = 1.11; 11,006 km gives far-from-home = 1	**Pass**
Training	Train on HDFS data and save the model	3.2 min run; model folders `metadata` and `stages` in HDFS	**Pass**
Stage 3	Score a stream and write to MySQL	5,000 messages sent, 5,000 rows in MySQL, none lost	**Pass**
Dashboard	Live update while streaming	Figures rose from 5,000 to 8,000 during a run	**Pass**
Persistence	Rebuild the Spark and Kafka containers	Dataset, model and MySQL data intact afterwards	**Pass**
Final run	Clean end-to-end run	10,000 transactions scored and displayed	**Pass**

7.2 Challenges and how we solved them

Running seven Big Data services on one 8 GB laptop is demanding, and several problems had to be solved along the way. We document them because they are the kind of problems any team deploying this stack will meet.

Table 7.2: Problems encountered and their solutions

Problem	Cause	Solution
Docker engine stuck on start-up	A Docker Desktop update switched the engine from WSL2 to Hyper-V, where our memory settings did not apply	Switched back to the WSL2 engine and restarted cleanly
Data and topics lost	The engine switch recreated Docker's storage	Rebuilt HDFS, the topic and the scripts; later made all state persistent (named volumes, custom image, shared folders)
Spark job failed before starting	The container could not reach the Maven repository to download the Kafka connector	Downloaded the JARs on Windows and passed them with `--jars`
MLlib and model scoring crashed	The official Spark image lacks numpy and pandas	Installed them, then built them into a custom image
Training froze Docker for two hours	Docker had only 2 GB of memory for seven containers plus training	Raised Docker to 3 GB, paused unused containers during training, trained on a 5% sample, and kept only needed columns in memory. Training then took 3 minutes
Weak first model (AUC 0.63)	The first dataset was mostly random	Measured the best achievable score and regenerated the data (Section 6.2)
Repeated transactions in demos	The producer always started at the top of the file	Added `--start-row`; reset the checkpoint and table before the final run
Lost executors in long runs	The laptop went to sleep while Spark was running	Keep the machine awake; checkpointing lets the stream resume

Lesson learned: memory is the real bottleneck on a laptop cluster

Every service in the stack runs on the Java virtual machine, and each one reserves memory as it starts. On a laptop, the question is not whether the code is correct, but whether all services fit in memory at the same time. Pausing services a job does not need, and giving Spark only the columns it actually uses, made the difference between a job that froze the machine and one that finished in three minutes.

7.3 Deployment

The dashboard is served by Django at `http://127.0.0.1:8000`. To make it available to other people on the same network, for example during the presentation, Django can listen on all network interfaces:

```
python manage.py runserver 0.0.0.0:8000
```

Anyone on the network can then open `http://<laptop-IP-address>:8000`. For a permanent public deployment, the same Django project would run behind a production web server (for example Gunicorn and Nginx) on a cloud virtual machine, connected to AWS RDS instead of the local MySQL container (Chapter 9).

Chapter 8

Setup and Execution Guide

This chapter lets anyone rebuild and run the complete system from the project folder. All commands are for the Windows command prompt, run from the project folder unless stated otherwise.

8.1 Requirements

- Windows 10 or 11 with WSL2, and Docker Desktop using the WSL2 engine
- Python 3.10 or newer on Windows
- 8 GB of RAM (3 GB given to Docker) and about 15 GB of free disk space

8.2 Project structure

```
BIG DATA ESSENTIAL PROJECT/
  docker-compose.yml          all seven services
  hadoop.env                  HDFS configuration
  spark/Dockerfile            custom Spark image (numpy, pandas)
  scripts/                    generate_fraud_dataset.py, create_tables.sql
  data/                       transactions.csv (generated, 1.25 GB)
  producer/                   kafka_producer.py
  consumer/                   spark_consumer_stage1.py, _stage2.py, _stage3.py
  models/                     train_fraud_model.py, training_metrics.json
  jars/                       the five JAR files
  django_dashboard/           Django project (fraud_dashboard, monitor)
```

8.3 Step by step

Step 1: Give Docker enough memory

Create or edit `%USERPROFILE%\.wslconfig`, then restart Docker Desktop after running `wsl --shutdown`:

```
[wsl2]
memory=3GB
processors=2
swap=4GB
```

Step 2: Start the cluster

```
docker-compose up -d --build
docker ps
```

Seven containers should be listed as “Up”. The first build takes a few minutes.

Step 3: Generate the dataset

```
cd scripts
python generate_fraud_dataset.py --target-gb 1.2 --out ../data/transactions.csv
cd ..
```

Step 4: Load the dataset into HDFS

```
docker cp data/transactions.csv namenode:/tmp/transactions.csv
docker exec -it namenode hdfs dfs -mkdir -p /fraud_project/data
docker exec -it namenode hdfs dfs -put /tmp/transactions.csv /fraud_project/data/
docker exec -it namenode hdfs dfs -ls -h /fraud_project/data/
docker exec -it namenode rm /tmp/transactions.csv
```

Step 5: Create the Kafka topic and the MySQL table

```
docker exec -it kafka kafka-topics --create --topic transactions_topic --bootstrap-server
    ↪ localhost:9092 --partitions 1 --replication-factor 1
docker exec -i mysql mysql -ubde_user -pbde_pw fraud_db < scripts\create_tables.sql
```

Step 6: Download the JAR files (once)

```
curl -L -o jars\spark-sql-kafka-0-10_2.12-3.5.3.jar https://repo1.maven.org/maven2/org/apache/
    ↪ spark/spark-sql-kafka-0-10_2.12/3.5.3/spark-sql-kafka-0-10_2.12-3.5.3.jar
curl -L -o jars\spark-token-provider-kafka-0-10_2.12-3.5.3.jar https://repo1.maven.org/maven2/
    ↪ org/apache/spark/spark-token-provider-kafka-0-10_2.12/3.5.3/spark-token-provider-kafka
    ↪ -0-10_2.12-3.5.3.jar
curl -L -o jars\kafka-clients-3.4.1.jar https://repo1.maven.org/maven2/org/apache/kafka/kafka-
    ↪ clients/3.4.1/kafka-clients-3.4.1.jar
curl -L -o jars\commons-pool2-2.11.1.jar https://repo1.maven.org/maven2/org/apache/commons/
    ↪ commons-pool2/2.11.1/commons-pool2-2.11.1.jar
curl -L -o jars\mysql-connector-j-8.0.33.jar https://repo1.maven.org/maven2/com/mysql/mysql-
    ↪ connector-j/8.0.33/mysql-connector-j-8.0.33.jar
```

Step 7: Train the model

Pause the services training does not need, train, then start them again. Results are written to `models\training_metrics.json` and the model to HDFS.

```
docker stop kafka zookeeper mysql
docker exec -it spark-master /opt/spark/bin/spark-submit --driver-memory 512m --executor-memory
    ↪ 768m /tmp/models/train_fraud_model.py --sample-fraction 0.05
docker start zookeeper kafka mysql
```

Step 8: Start real-time scoring (terminal 1, keep it open)

```
docker exec -it spark-master /opt/spark/bin/spark-submit --driver-memory 512m --executor-memory
    ↪ 768m --jars /tmp/jars/spark-sql-kafka-0-10_2.12-3.5.3.jar,/tmp/jars/spark-token-provider-
    ↪ kafka-0-10_2.12-3.5.3.jar,/tmp/jars/kafka-clients-3.4.1.jar,/tmp/jars/commons-pool2
    ↪ -2.11.1.jar,/tmp/jars/mysql-connector-j-8.0.33.jar /tmp/consumer/spark_consumer_stage3.
    ↪ py
```

Wait for the message “Stage 3 consumer started”.

Step 9: Start the dashboard (terminal 2, keep it open)

```
cd django_dashboard
pip install -r requirements.txt
python manage.py runserver
```

Open `http://127.0.0.1:8000` in a browser.

Step 10: Stream transactions (terminal 3)

```
cd producer
pip install kafka-python requests
python kafka_producer.py --max-rows 10000 --delay 0.01
```

The dashboard fills up within seconds. For another run with new transactions, add `--start-row 10000`.

8.4 Starting over with a clean slate

To remove all previous predictions before a demonstration (stop terminal 1 first, then start it again afterwards):

```
docker exec -it namenode hdfs dfs -rm -r -skipTrash /fraud_project/checkpoints/stage3
docker exec -it mysql mysql -ubde_user -pbde_pw fraud_db -e "TRUNCATE TABLE fraud_predictions;"
```

8.5 Troubleshooting

Symptom	Fix
Docker Desktop shows RAM 0.00 GB or commands return error 500	Docker is out of memory. Quit Docker Desktop, run `wsl --shutdown`, reopen it, and close heavy applications.
`No module named 'numpy'` **or** `'pandas'`	The containers were not built from `spark/Dockerfile`. Run `docker-compose up -d --build`.
Spark cannot find the Kafka source	A JAR is missing from `jars\`. Repeat Step 6.
Dashboard says it cannot reach the database	The `mysql` container is not running. Check with `docker ps`.
Stage 3 fails after the Kafka topic was recreated	The checkpoint points to the old topic. Delete it (clean-slate commands above).

Chapter 9

Limitations and Future Work

9.1 Limitations

- **Synthetic data.** Our dataset follows known fraud patterns, which made it possible to measure the best achievable score, but real fraud is messier and fraudsters adapt. Results on real bank data would be lower.
- **Labels arrive instantly.** In our stream the true label travels with each transaction. In reality a bank learns that a transaction was fraud days or weeks later, usually through a customer complaint or chargeback.
- **Customer history is given, not computed.** Values such as the 30-day average spend come with each transaction. A production system would compute them itself from the stream.
- **A single-machine cluster.** HDFS, Kafka and Spark each run as one node with replication 1. The design is distributed, but the fault tolerance and speed-up of a real cluster were not tested.
- **Development server.** The dashboard runs on Django's built-in server, which is intended for development.

9.2 Future work

- **Move the database to AWS RDS.** The guidelines award extra credit for AWS RDS. Our code is ready for it: the Spark job would only need a new JDBC address, and the Django settings already read the database host, user and password from environment variables.
- **Compute behaviour features in the stream.** Use Spark's stateful, windowed aggregations to calculate each customer's recent spending and transaction velocity live, instead of reading them from the data.
- **Cost-based thresholds.** Choose the HIGH and MEDIUM thresholds by weighing the cost of a missed fraud against the cost of a review, and let analysts adjust them from the dashboard.
- **Stronger models and retraining.** Try gradient-boosted trees (`GBTClassifier` in MLlib), and retrain the model regularly as confirmed fraud labels arrive, monitoring for changes in fraud behaviour.
- **Alerts that reach people.** Send HIGH-risk alerts to analysts by e-mail or SMS, and add a login to the dashboard.

Chapter 10

Conclusion

We set out to build a system that catches card fraud while it is happening, and we built it: transactions are read from HDFS, streamed through Kafka, enriched and scored by Spark with a Random Forest trained in MLlib, stored in MySQL and shown on a Django dashboard within seconds, all running together on a single laptop.

The numbers hold up. The Random Forest reaches an AUC-ROC of 0.904 and catches 82% of frauds on unseen test data, essentially the best score this data allows, and it behaves the same way on the live stream. The findings also go beyond the numbers: fraud is revealed by *combinations* of warning signals, the features we engineered carry most of the predictive power, and the quality of the data sets a hard limit that no algorithm can overcome. Measuring that limit, rather than blindly tuning the model, was one of the most valuable things we did.

Finally, the project taught us that Big Data engineering is as much about making components cooperate (versions that match, memory that fits, state that survives a restart) as it is about algorithms. The result is a working, documented and reproducible pipeline, and a solid base for the next steps: real data, AWS RDS and features computed live from the stream.

References

[1] K. Kumar, *Big Data Essentials Course Project Guidelines for Students: Integrate Big Data Technologies for Prediction and Real-Time Analytics*. Adventist University of Central Africa, May 2026.

[2] K. Shvachko, H. Kuang, S. Radia and R. Chansler, "The Hadoop Distributed File System," in *Proc. IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, 2010.

[3] J. Kreps, N. Narkhede and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, 2011.

[4] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[5] M. Armbrust *et al.*, "Structured Streaming: A declarative API for real-time applications in Apache Spark," in *Proc. ACM SIGMOD*, 2018.

[6] X. Meng *et al.*, "MLlib: Machine learning in Apache Spark," *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.

[7] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[8] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. 23rd International Conference on Machine Learning (ICML)*, 2006.

[9] A. Dal Pozzolo, O. Caelen, R. A. Johnson and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *Proc. IEEE Symposium Series on Computational Intelligence (SSCI)*, 2015.

[10] Apache Software Foundation, *Structured Streaming + Kafka Integration Guide (Spark 3.5.3)*. `https://spark.apache.org/docs/3.5.3/structured-streaming-kafka-integration.html`

[11] Django Software Foundation, *Django 4.2 Documentation*. `https://docs.djangoproject.com/en/4.2/`

Appendix A

Code Highlights

A.1 The shared feature recipe

This function appears, identically, in the training script and in the Stage 2 and Stage 3 streaming consumers.

```
def clean_and_add_features(df):
    return df \
        .dropna(subset=["transaction_id", "timestamp", "amount", "avg_txn_amount_last_30d"]) \
        .filter(col("amount") > 0) \
        .withColumn("event_time", to_timestamp(col("timestamp"), "yyyy-MM-dd HH:mm:ss")) \
        .dropna(subset=["event_time"]) \
        .withColumn("amount_to_avg_ratio",
                    when(col("avg_txn_amount_last_30d") > 0,
                         spark_round(col("amount") / col("avg_txn_amount_last_30d"), 2))
                    .otherwise(spark_round(col("amount"), 2))) \
        .withColumn("hour_of_day", hour(col("event_time"))) \
        .withColumn("is_high_risk_merchant",
                    when(col("merchant_category").isin(["jewelry", "electronics"]), 1).otherwise
    ↪ (0)) \
        .withColumn("is_far_from_home", when(col("distance_from_home_km") > 500, 1).otherwise(0)
    ↪ ) \
        .withColumn("is_high_velocity", when(col("transactions_last_1h") >= 3, 1).otherwise(0))
    ↪ \
        .withColumn("card_present_int", col("card_present").cast("int"))
```

A.2 Scoring each micro-batch and writing it to MySQL

```
model = PipelineModel.load("hdfs://namenode:9000/fraud_project/models/fraud_model")

scored = model.transform(clean_and_add_features(parsed_stream)) \
    .withColumn("fraud_probability", spark_round(vector_to_array(col("probability"))[1], 4)) \
    .withColumn("risk_level",
                when(col("fraud_probability") >= 0.80, "HIGH")
                .when(col("fraud_probability") >= 0.50, "MEDIUM")
                .otherwise("LOW"))

scored.writeStream \
    .foreachBatch(write_batch_to_mysql) \
    .option("checkpointLocation", "hdfs://namenode:9000/fraud_project/checkpoints/stage3") \
    .trigger(processingTime="5 seconds") \
    .start()
```

A.3 The fraud formula of the dataset generator (v2)

```
risk = (3.5 * spending_spike          # amount > 3 x 30-day average
      + 3.5 * far_from_home           # more than 500 km from home
      + 2.8 * burst_of_transactions   # 3 or more in the last hour
      + 1.2 * jewelry + 0.8 * electronics
      + 0.6 * card_not_present)
fraud_probability = clip(1 / (1 + exp(-(risk - 7.0))), 0.0002, 0.97)
```